

Scenario:

The e-commerce company wants to maintain information about products and their categories. Each product belongs to a specific category, and the system should manage product details such as name, price, and stock availability.

1. Design Product and Category Tables

Category Table:

```
SQL> CREATE TABLE CATEGORY(CATEGORY_ID VARCHAR(7),CATEGORY_NAME VARCHAR(20),  
DESCRIPTION VARCHAR(50));
```

Table created.

Product Table:

```
SQL> CREATE TABLE PRODUCT(PRODUCT_ID VARCHAR(7),PRODUCT_NAME VARCHAR(20),CA  
TEGORY_ID VARCHAR(7),PRICE NUMBER(9,2),STOCK_QUANTITY NUMBER);
```

Table created.

The two tables for Category and Product have been created.

Now, we have to find the relationship between the tables and apply constraints to the tables.

2. Define Relationships and Constraints

Relationships:

One Category Can have many Products.

The relationship between Category and Product is One to many (1-> *).

We found that, what is their relationship. Now we have to discuss that which is the key that is going to make this relationship.

Before that, we need to understand about KEY Constraints.

PRIMARY KEY:

The Column(s) or Field(s) that uniquely identifies a row / tuple

In these two tables, which field can be a Primary Key?

In Category Table, Category_ID is a unique Value that helps to identify each category. Likewise, in Product Table Product_ID is a unique Column.

FOREIGN KEY:

The Key which links two tables.

Therefore, Category_ID acts as Foreign Key in the Product Table.

We can explore some other constraints like NOT NULL, CHECK and UNIQUE

NOT NULL:

Ensures that the Column cannot have a empty (null) value.

For Example, a Product name or Category name cannot be empty. All category or product must have a name.

CHECK:

Validates the values in a column satisfies the condition.

For Example, To check the Price of a Product must always be a positive number.

UNIQUE:

Ensures that there is no repeated value for a column.

For Example, If we mention UNIQUE Constraint to Category Name, no two category can have same name.

Defining Relationship and Creating Constraints for Product and Category:

Primary Key:

```
SQL> ALTER TABLE CATEGORY ADD CONSTRAINT PK_CATEGORY PRIMARY KEY (CATEGORY_ID);  
Table altered.  
  
SQL> ALTER TABLE PRODUCT ADD CONSTRAINT PK_PRODUCT PRIMARY KEY (PRODUCT_ID);  
Table altered.
```

Creating Relationship using FOREIGN KEY:

```
SQL> ALTER TABLE Product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(Category_id);  
Table altered.
```

Adding Constraints:

NOT NULL:

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;
Table altered.

SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;
Table altered.
```

CHECK:

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_PRICE CHECK (Price > 0);
Table altered.
```

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);
Table altered.
```

UNIQUE:

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category UNIQUE (Category_Name);
Table altered.
```

3. Insert Values:

```
SQL> INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description');
Enter value for category_id: 1
Enter value for category_name: Roses
Enter value for description: Fresh rose flowers
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('1','Roses','Fresh rose flowers')

1 row created.

SQL> /
Enter value for category_id: 2
Enter value for category_name: Lilies
Enter value for description: Fresh lily flowers
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('2','Lilies',' Fresh lily flowers')

1 row created.

SQL> /
Enter value for category_id: 3
Enter value for category_name: Jasmine
Enter value for description: Fresh jasmine flowers
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('3',' Jasmine','Fresh jasmine flowers')

1 row created.

SQL> /
Enter value for category_id: 4
Enter value for category_name: Orchids
Enter value for description: Decorative orchid flowers
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('4','Orchids','Decorative orchid flowers')

1 row created.

SQL> /
Enter value for category_id: 5
Enter value for category_name: Bouquets
Enter value for description: Handmade flower bouquets
```

Product Table:

```
SQL> INSERT INTO Product VALUES('&Product_ID','&Product_Name',&Category_ID,&Price,&Stock_Quantity);
Enter value for product_id: 11
Enter value for product_name: Red Rose
Enter value for category_id: 1
Enter value for price: 50
Enter value for stock_quantity: 100
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name',&Category_ID,&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('11','Red Rose',1,50,100)

1 row created.

SQL> /
Enter value for product_id: 12
Enter value for product_name: White Lily
Enter value for category_id: 2
Enter value for price: 80
Enter value for stock_quantity: 60
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name',&Category_ID,&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('12',' White Lily',2,80,60)

1 row created.

SQL> /
Enter value for product_id: 13
Enter value for product_name: Jasmine
Enter value for category_id: 3
Enter value for price: 40
Enter value for stock_quantity: 150
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name',&Category_ID,&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('13',' Jasmine',3,40,150)

1 row created.
```

```
SQL> /
Enter value for product_id: 14
Enter value for product_name: Purple Orchid
Enter value for category_id: 4
Enter value for price: 120
Enter value for stock_quantity: 50
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name',&Category_ID,&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('14','Purple Orchid',4,120,50)

1 row created.

SQL> /
Enter value for product_id: 15
Enter value for product_name: Rose Bouquet
Enter value for category_id: 5
Enter value for price: 350
Enter value for stock_quantity: 30
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name',&Category_ID,&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('15',' Rose Bouquet',5,350,30)

1 row created.
```

Like this insert few more rows.

After Insertion Category Table:

```
SQL> SELECT * FROM CATEGORY;
```

CATEGORY_ID	CATEGORY_NAME	DESCRIPTION
1	Roses	Fresh rose flowers
2	Lilies	Fresh lily flowers
3	Jasmine	Fresh jasmine flowers
4	Orchids	Decorative orchid flowers
5	Bouquets	Handmade flower bouquets

```
SQL> SELECT * FROM PRODUCT;
```

PRODUCT_ID	PRODUCT_NAME	CATEGORY_ID	PRICE	STOCK_QUANTITY
11	Red Rose	1	50.00	100
12	White Lily	2	80.00	60
13	Jasmine	3	40.00	150
14	Purple Orchid	4	120.00	50
15	Rose Bouquet	5	350.00	30

Product Table:CRUD Operations:

Insert Record:

```
SQL> INSERT INTO PRODUCT (PRODUCT_ID, PRODUCT_NAME, PRICE, STOCK_QUANTITY)
2 VALUES ('16', 'Marigold', 60, 90);

1 row created.
```

Update Price:

```
SQL> UPDATE Product SET Price = 550 WHERE PRODUCT_ID = 13;

1 row updated.
```

Delete a Product:

```
SQL> DELETE FROM Product WHERE Product_Name = 'Marigold';

1 row deleted.
```

5. Generate Category-wise Product Reports

Write SQL queries to:

- Display all products under each category.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price, p.Stock_Quantity FROM Pro
duct p JOIN Category c ON p.Category_ID=c.Category_ID;
```

CATEGORY_NAME	PRODUCT_NAME	PRICE	STOCK_QUANTITY
Roses	Red Rose	50.00	100
Lilies	White Lily	80.00	60
Jasmine	Jasmine	550.00	150
Orchids	Purple Orchid	120.00	50
Bouquets	Rose Bouquet	350.00	30

- Count number of products in each category.

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	PRODUCT_COUNT
Roses	1
Jasmine	1
Bouquets	1
Orchids	1
Lilies	1

- Find highest-priced product in each category.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price AS Highest_price FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID WHERE p.Price=(SELECT MAX(p2.Price) FROM Product p2 WHERE p2.Category_ID=p.Category_ID);
```

CATEGORY_NAME	PRODUCT_NAME	HIGHEST_PRICE
Roses	Red Rose	50
Lilies	White Lily	80
Jasmine	Jasmine	550
Orchids	Purple Orchid	120
Bouquets	Rose Bouquet	350

- Display categories having more than 2 products.

```
SQL> SELECT c.Category_Name FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID GROUP BY c.Category_Name HAVING COUNT(p.Product_ID)>2;
```

no rows selected

- Find average product price category-wise.

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Avg_Price FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	AVG_PRICE
Roses	50
Jasmine	550
Bouquets	350
Orchids	120
Lilies	80

